02 iDeliveryFood Prompts Fundacao Seguranca Entrega

Regras desta revisão — aplicação integral

Este documento conserva o conteúdo e os módulos do PDF anexado, com alterações nas instruções afetadas, sem resumo. As regras abaixo se aplicam a todos os módulos e critérios de aceite deste arquivo.

Mobile nativo e testes por plataforma

Android: Kotlin e Jetpack Compose; iOS: Swift e SwiftUI, projetos separados. Sem app híbrido/React. Implementar e testar cada plataforma independentemente: unitários de domínio e ViewModel, integração API, UI, deep links, sessão, permissões, offline e acessibilidade. CI Android com toolchain apropriado; CI iOS em runner macOS com Xcode. Assinatura e contas de distribuição são externas ao Git. Falta de ambiente Apple deve aparecer como pendência, nunca como build/teste concluído. Web/PWA permanece entrega web separada, jamais substitui os apps nativos.

MCP exato fornecido pelo proprietário

{
"mcpServers": {
"supabase": {
"serverUrl": "https://mcp.supabase.com/mcp?project_ref=dwsoochnfxldfvcaijie&features=docs%2Caccount%2Cdatabase%2Cdebugging%2Cdevelopment%2Cfunctions%2Cbranching%2Cfeedback"
}
}
}

SQL remoto e seed

SUPABASE — PROJETO CORRIGIDO E EXECUÇÃO REMOTA: projeto autorizado dwsoochnfxldfvcaijie. Configure o MCP com o JSON exato desta revisão e autenticação segura. Inspecione o schema, crie/altere tabelas, índices, constraints, funções, RLS, grants, Storage e demais recursos necessários; execute os SQLs no Supabase por migrations versionadas e verifique o resultado real. Não entregue somente arquivos SQL sem aplicá-los. A senha anteriormente exposta deve ser rotacionada; nunca reproduzi-la ou versioná-la. Não gravar tokens no Git. Identifique o ambiente antes de inserir seed: usar desenvolvimento/teste. Em produção ou ambiente desconhecido, confirmar antes de gravar dados sintéticos. Não executar DROP/TRUNCATE/reset remoto ou alterações destrutivas sem aprovação específica. Registrar migration, resultado, validação e plano de recuperação sem segredos.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

GitHub depois dos testes

O Antigravity deve executar commit e push em https://github.com/JuanBMMontesinos/iDeliveryFood.git depois de testar o código e revisar diff e segredos. Verificar branch/remotes e preservar histórico; não usar force-push, mudar visibilidade ou contornar proteção. Respeitar gates de aprovação do ambiente e autenticar por mecanismo seguro. Executar testes locais antes do push e acompanhar CI remoto depois; falha de CI requer correção e novo push, sem alegar release aprovado. Confirmar SHA remoto. Push não é deploy nem publicação nas lojas.

Idioma principal

INTERFACE E i18n: inglês (en) como idioma principal e padrão inicial; usuário pode mudar para português brasileiro (pt-BR), com preferência persistida no Supabase para conta autenticada. Android usa recursos strings/plurals, iOS usa String Catalog/localization, web usa solução i18n com chaves, parâmetros, pluralização e fallback en. Não concatenar frases traduzíveis; traduzir validações, erros públicos, acessibilidade e notificações. Formatar datas/números/moeda por locale; BRL e região de operação não mudam só porque a interface está em inglês. Responsividade e acessibilidade; estados loading/vazio/erro/offline/sem permissão. Testar troca de idioma, persistência, fallback e paridade de chaves. O termo técnico é i18n (não i8n).

Documentação e segurança preservadas

Manter documentação completa, testes unitários, integração, RLS/pgTAP, e2e e controles de cybersegurança dos módulos originais. Atualizar arquitetura, ERD, migrations aplicadas, seed, API, matriz de permissões, instruções nativas de build/teste, i18n, CI/CD, operação e evidências. Não incluir segredos; não declarar certificação de segurança sem validação independente.

Conteúdo integral revisado do documento anexado

iDeliveryFood — Fundação, segurança e

entrega

Como usar

PACOTE REVISADO — PROMPTS POR APLICATIVO, PORTAL E MÓDULO

Este material substitui o pacote anterior. É uma especificação e conjunto de comandos para o agente construir o iDeliveryFood incrementalmente; não é código já implementado nem certificação de produção.

Os prompts cobrem explicitamente app web e app mobile do cliente, app mobile do entregador, portal do parceiro, gestor de pedidos, portal de redes, console logístico, área de frota, administração central e backoffices financeiro, atendimento, comercial, marketing, relatórios, risco e privacidade, além da infraestrutura comum.

Como usar: cole as instruções permanentes antes do prompt do módulo. Em uma nova sessão, repita-as e peça ao agente para ler docs/DELIVERY_STATUS.md e os contratos existentes. Execute um módulo por tarefa, respeitando dependências. Não envie todos para execução simultânea. IDs indicam módulos, não obrigatoriamente a ordem numérica.

Projeto Supabase corrigido: dwsoochnfxldfvcaijie. Executar migrations e seed no ambiente de desenvolvimento autorizado conforme esta revisão. Rotacionar a senha exposta; arquivos não contêm senha nem URI credenciada.

Não há publicação GitHub, acesso ao banco ou implantação realizada por este pacote. Contas externas, contratação de serviços, credenciais seguras, revisão jurídica e aprovações permanecem responsabilidade do proprietário.

Instruções permanentes

INSTRUÇÕES PERMANENTES — COPIAR COMO CONTEXTO EM TODAS AS SESSÕES

Você é a equipe de engenharia do iDeliveryFood dentro do Antigravity. Execute uma etapa por vez e produza código funcional, não apenas planos, protótipos ou descrições. Inspecione o repositório e siga os contratos/ADRs existentes. Não substitua stack ou reescreva módulos prontos sem justificar e obter aprovação. Identifique esta entrega pelo ID do módulo, mantenha docs/DELIVERY_STATUS.md atualizado e não afirme conclusão sem evidências.

REPOSITÓRIO: alvo solicitado pelo proprietário:

https://github.com/JuanBMMontesinos/iDeliveryFood.git. Verifique primeiro acesso, conteúdo, branch,

remotes e estado local. Não suponha que exista ou esteja vazio. Após testes aprovados, executar commit e push ao repositório solicitado e aplicar migrations no Supabase autorizado. Respeitar aprovações do ambiente; não alterar visibilidade, usar force-push ou executar operações destrutivas sem autorização específica. Antes de qualquer

publicação: revisar diff, testes e secret scan; não expor credenciais ou dados pessoais.

SUPABASE — PROJETO CORRIGIDO E EXECUÇÃO REMOTA: projeto autorizado dwsoochnfxldfvcaijie. Configure o MCP com o JSON exato desta revisão e autenticação segura. Inspecione o schema, crie/altere

tabelas, índices, constraints, funções, RLS, grants, Storage e demais recursos necessários; execute os SQLs no Supabase por migrations versionadas e verifique o resultado real. Não entregue somente arquivos SQL sem aplicá-los. A senha anteriormente exposta deve ser rotacionada; nunca reproduzi-la ou versioná-la. Não gravar tokens no Git. Identifique o ambiente antes de inserir seed: usar desenvolvimento/teste. Em produção ou ambiente desconhecido, confirmar antes de gravar dados sintéticos. Não executar DROP/TRUNCATE/reset remoto ou alterações destrutivas sem aprovação específica. Registrar migration, resultado, validação e plano de recuperação sem segredos.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

ARQUITETURA DE REFERÊNCIA: monorepo com backend/web TypeScript e workspace pnpm para apps/customer-web (Next.js), apps/partner-web (Next.js), apps/admin-web (Next.js), apps/api (NestJS) e apps/worker. Mobile com código NATIVO separado: apps/customer-android e apps/courier-android em Kotlin/Jetpack Compose; apps/customer-ios e apps/courier-ios em Swift/SwiftUI. Se algum novo app mobile for habilitado, também terá implementações Android e iOS independentes. NÃO usar React Native, Expo, Flutter, Ionic, Capacitor ou solução híbrida; não encapsular páginas web como app. Compartilhar contratos OpenAPI e especificações de domínio, não UI/runtime TypeScript com os apps. packages/contracts, domain, authz, api-client, ui-web, config são para backend/web; clientes nativos próprios por plataforma. Gradle e Xcode gerenciam seus projetos. Validar versões estáveis, lockfiles e ADR de migração se já houver código. Logística e portais de rede/frota mantêm áreas web autorizadas próprias; um backend comum para todas as interfaces.

CONTRATOS: API versionada /v1 com OpenAPI e cliente tipado; Supabase Auth para identidade; PostgreSQL/Supabase como fonte de verdade; AuthZ no servidor e RLS/grants no banco para as superfícies acessíveis; Realtime somente em canais autorizados. Negócio e escrita privilegiada no backend; consultas diretas do cliente apenas quando explicitamente aprovadas e protegidas por RLS. Service-role e segredos exclusivamente no servidor, jamais bundles/public env. Roles não podem vir de metadata editável pelo usuário. Multi-tenant por organização e unidade; evitar caches compartilhados entre tenants. Finanças, auditoria e documentos sensíveis em superfícies restritas.
SEGURANÇA: MFA obrigatório para staff privilegiado; cookies/tokens apropriados a cada plataforma, validação server-side, CORS allowlist, CSP e headers na web, rate limits, proteção a CSRF/XSS/SQLi/SSRF/IDOR, uploads seguros, rotação/revogação, log redigido sem segredos/PII indevida. Mobile usa armazenamento seguro de tokens e permissões mínimas. Não armazenar dados de cartão. Endereço/localização/documentos com exposição e retenção mínimas. Mudança bancária, repasses e reembolsos acima da alçada exigem controle adicional. Threat model e checklist OWASP ASVS versionados como metas verificáveis, sem alegar certificação. Revisão jurídica e pentest externo continuam pendências de lançamento.
PADRÕES DE DOMÍNIO: centavos inteiros para BRL; datas persistidas UTC e exibidas

America/Sao_Paulo; validação de timezone de horário de loja; UUIDs/IDs não substituem autorização; transações, constraints, índices e idempotency keys para mutações críticas; outbox e consumidores idempotentes para tarefas assíncronas. Máquina de estados e histórico de eventos de pedido separados dos estados de pagamento e entrega. Não chamar provedor externo dentro de transação longa. Estado final do PSP exige validação de assinatura, correlação e reconciliação. Não confiar em preço, comissão, papel ou status enviado pelo cliente.

INTERFACE E i18n: inglês (en) como idioma principal e padrão inicial; usuário pode mudar para português brasileiro (pt-BR), com preferência persistida no Supabase para conta autenticada. Android usa recursos strings/plurals, iOS usa String Catalog/localization, web usa solução i18n com chaves, parâmetros, pluralização e fallback en. Não concatenar frases traduzíveis; traduzir validações, erros públicos, acessibilidade e notificações. Formatar datas/números/moeda por locale; BRL e região de operação não mudam só porque a interface está em inglês. Responsividade e acessibilidade; estados loading/vazio/erro/offline/sem permissão. Testar troca de idioma, persistência, fallback e paridade de chaves. O termo técnico é i18n (não i8n).

QUALIDADE: cada módulo traz testes unitários de regras, integração local com Supabase, casos de acesso negado e multi-tenant, contratos e e2e aplicáveis. CI exige lint, typecheck, testes, RLS/pgTAP, build e secret scan. Propor meta inicial de cobertura de 80% em código de domínio e cobertura de decisões críticas, sem manipular métricas; não é resultado garantido. Testes ausentes ou pulados explicitamente relatados. Nunca executar testes com cobrança ou dados sintéticos em produção.

FINAL DE CADA ETAPA: listar arquivos alterados; migrations; endpoints; telas; permissões; comandos executados e resultados reais; cobertura; evidências; riscos; pendências de configuração; atualização de docs e próximos passos. Se faltar credencial/aprovação, continuar apenas no trabalho seguro local e registrar BLOQUEADO, não CONCLUÍDO. Após os testes e secret scan, realizar commit e push no GitHub alvo, respeitando gates do ambiente e confirmando SHA remoto.

M00 — Auditoria, produto e contratos compartilhados

PROMPT M00 — Auditoria, produto e contratos compartilhados

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: Início obrigatório.

OBJETIVO: Estabelecer a base comum antes de qualquer app.

IMPLEMENTAÇÃO E TELAS

Inspecione workspace e Git sem ler/imprimir segredos. Produza PRD, escopo por fase, matriz de apps/atores, inventário de integrações, decisões abertas e ADR de stack. Defina fontes de verdade, limites dos módulos, convenções de IDs/dinheiro/tempo/paginação/erros e contrato OpenAPI inicial. Documente que marketplace de comida é o núcleo; mercado, publicidade e assinatura são extensões condicionadas.

Decisões bloqueadoras: projeto Supabase correto; ambiente dev/stage/prod; modalidade

loja/plataforma/híbrida; região; PSP/split; mapas; comunicação; política de taxas, cancelamento, repasse e cadastro dos entregadores. Proponha padrões, não invente aprovação.

DADOS, CONTRATOS E ACESSOS

Documentos e schemas de contratos; sem escrita remota ou dados de domínio.

TESTES OBRIGATÓRIOS

Validar schemas e consistência dos contratos; identificar comandos reproduzíveis e ferramentas disponíveis.

CRITÉRIOS DE ACEITE

Plano com dependências, riscos, critérios de aceite, decisões aprovadas/abertas e nenhum segredo no repositório.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M00.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M01 — Monorepo, ambientes e desenvolvimento local

PROMPT M01 — Monorepo, ambientes e desenvolvimento local

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M00.

OBJETIVO: Criar estrutura executável para todos os aplicativos.

IMPLEMENTAÇÃO E TELAS

Crie apps e packages da arquitetura aprovada, workspace pnpm, lint/format/typecheck, lockfile, configuração de ambiente validada por app, logging redigido e health endpoints reais. Instale Supabase local e scripts de reset somente local com bloqueio explícito para host remoto. Separar .env por aplicativo; .env.example só nomes/placeholders. Diferenciar build-time/runtime env e variáveis públicas. Crie esqueletos sem telas de demonstração e sem dados falsos. Integrações ausentes devem permanecer indisponíveis.

DADOS, CONTRATOS E ACESSOS

Configuração de ambiente e health, com supabase/seed.sql fictício, idempotente, persistido no Supabase de desenvolvimento/teste e sem dados chumbados nas interfaces. Definir conexão server e client

separadas.

TESTES OBRIGATÓRIOS

Build de cada app, lint, typecheck e teste de validação de env sem expor valores; garantir secrets ausentes em bundles.

CRITÉRIOS DE ACEITE

Todos os apps iniciam localmente ou explicam exatamente dependências bloqueadas; setup reproduzível por README.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M01.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M02 — Supabase: banco, migrations, Auth, RLS e Storage

PROMPT M02 — Supabase: banco, migrations, Auth, RLS e Storage

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M00, M01; projeto dwsoochnfxldfvcaijie; executar migrations e seed no Supabase autorizado.

OBJETIVO: Implementar a fundação de dados e isolamento.

IMPLEMENTAÇÃO E TELAS

Desenhe ERD/dicionário para profiles, organizations/members, stores/hours/service_areas, catálogo, endereços, pedidos/itens/eventos, entregas, referências de pagamento, ledger, suporte, promoções e auditoria. Adicione entidades por fase, evitando dependências circulares. snapshots imutáveis do pedido, constraints de valores e estados, índices/geoespaciais se aprovados.

Ative RLS e grants mínimos nas tabelas expostas; por operação e tenant, testando consumidor, parceiro, entregador e staff. Proteja views/RPCs e SECURITY DEFINER com search_path e permissões explícitos. Buckets privados para documentos, políticas, MIME/tamanho, URLs curtas e rastreáveis. Catálogo público somente dados aprovados. Proteja escopo Realtime e subscriptions. Não alterar schema existente sem análise e plano de compatibilidade.

DADOS, CONTRATOS E ACESSOS

Migrations SQL versionadas, contratos de acesso, rollback/forward recovery; execução local antes de aplicar as migrations no Supabase autorizado, executando de fato os SQLs e verificando o resultado.

TESTES OBRIGATÓRIOS

pgTAP allow/deny por tabela/papel/ação, acesso cruzado entre organizações, RPC/view/storage, constraints e migration from clean local DB.

CRITÉRIOS DE ACEITE

ERD, dicionário, matriz RLS, migrations e evidências locais; nenhuma política allow-all para contornar erros.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M02.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M03 — Identidade, contas, permissões e onboarding

PROMPT M03 — Identidade, contas, permissões e onboarding

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M02.

OBJETIVO: Autenticar e autorizar todos os aplicativos sem bypass.

IMPLEMENTAÇÃO E TELAS

Implemente cadastro/login/logout/recuperação e verificação de contato via Supabase Auth conforme canais configurados. Convites para equipes com expiração, aceite auditado e escopo organização/unidade. MFA e step-up para funções privilegiadas. Aprovação de loja/entregador não confere papel editável pelo próprio usuário. Onboarding inicial do primeiro admin por procedimento local controlado, nunca endpoint público ou conta/senha fixa. Sessão web segura e tokens mobile em secure storage. Revogar sessão, dispositivos e acessos no desligamento. Fluxos de exportação/exclusão de conta com análise de retenção legal.

DADOS, CONTRATOS E ACESSOS

Profiles, memberships, staff scopes, invitations, approval events; contratos /me, membros e preferências.

TESTES OBRIGATÓRIOS

Login inválido, recuperação, expiração, revogação, tentativa de autoelevação de papel, multi-tenant, MFA staff e deep link mobile seguro.

CRITÉRIOS DE ACEITE

Permissões reais em API e banco; menu oculto não substitui autorização; nenhum login demo.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M03.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M04 — Catálogo, lojas, cobertura e preços

PROMPT M04 — Catálogo, lojas, cobertura e preços

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M02, M03.

OBJETIVO: Criar serviços de negócio reutilizados por cliente e parceiro.

IMPLEMENTAÇÃO E TELAS

Implemente cadastro de loja/categoria/produto/adicionais, regras de mínimo/máximo, disponibilidade, fotos autorizadas, horário normal/exceção, pausa e raio/polígono de cobertura conforme decisão. Publicação só de loja aprovada e operacional. Busca paginada com filtros reais e consulta geográfica eficiente. Serviço de cotação server-side com preço dos itens, adicionais, taxas, entrega e descontos aprovados. Validar endereço/cobertura/horário na cotação e novamente no pedido; versionar ou expirar cotação. Não inventar tempo de entrega ou taxa sem regra aprovada.

DADOS, CONTRATOS E ACESSOS

/stores, /catalog, /availability, /quotes; catálogo público separado de dados privados.

TESTES OBRIGATÓRIOS

Horário virando dia, feriados, fuso, fora de cobertura, produto pausado, adicionais inválidos, preço modificado, paginação e tenant.

CRITÉRIOS DE ACEITE

Busca/cotação retornam apenas dados reais; sem loja publicada mostrar vazio; preços calculados no servidor.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em

docs/modules/M04.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M10 — Pedidos, carrinho, cotação e máquina de estados

PROMPT M10 — Pedidos, carrinho, cotação e máquina de estados

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04.

OBJETIVO: Implementar o núcleo transacional antes das interfaces dependentes.

IMPLEMENTAÇÃO E TELAS

Defina estados separados de pedido, pagamento e entrega; documente tabela de transições/comandos/atores/efeitos. Criar pedido com cotação válida, snapshot de itens/preço/endereço necessário, idempotência e prevenção de duplicidade. Reserva de disponibilidade quando aplicável, rejeição/expiração e compensações. Modalidades retirada, entrega loja ou plataforma apenas se aprovadas. Confirmar pedido comercial e pagamento conforme fluxo escolhido; não pressupor pagamento aprovado ao criar order. Cancelamento parcial/total e elegibilidade calculados pelo backend e revisados conforme regras comerciais. Histórico append-only e outbox transacional para notificações/jobs. Tratar PSP aceito depois de expirar pedido como exceção conciliável, não sucesso silencioso.

DADOS, CONTRATOS E ACESSOS

/quotes, /orders, /orders/{id}/commands, /orders/{id}/events; enums publicados e versões controladas.

TESTES OBRIGATÓRIOS

Testes de transição completa, preço manipulado, idempotência, concorrência, expiração, compensação, timeout e rollback; isolamento usuário/loja.

CRITÉRIOS DE ACEITE

Nenhum estado impossível; efeitos financeiros/logísticos não duplicam; tabela de transições e testes correspondentes.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M10.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis

por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M23 — Cibersegurança transversal e auditoria técnica

PROMPT M23 — Cibersegurança transversal e auditoria técnica

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M01 em diante; gate antes do release.

OBJETIVO: Executar hardening verificável em todos os apps.

IMPLEMENTAÇÃO E TELAS

Produzir threat model de fronteiras web/mobile/API/banco/workers/PSP, inventário e checklist ASVS com versão e IDs verificados na documentação, requisito/evidência/status/risco. Revisar auth/MFA, RLS, endpoints, upload, SSRF/IDOR/injection, CSRF/CSP/CORS, secure storage mobile, deep links, secrets, dependências/SBOM, log redaction, isolamento cache/Realtime, rate limits distribuídos quando necessários, backup criptografado e incident response. CI com secret scan, SAST/dependency audit e análise de containers se usados. Não escanear infraestrutura externa sem autorização. Achado crítico/alto bloqueia release até resolução ou aceite formal de risco, não simples comentário. Risco residual deve ficar explícito; não alegar certificação, invulnerabilidade ou pentest não feito.

DADOS, CONTRATOS E ACESSOS

docs/security/threat-model.md, ASVS matrix, vulnerability register, disclosure/incident runbooks.

TESTES OBRIGATÓRIOS

Negativos cross-tenant de ponta a ponta, secrets em bundles/logs, replay, privilégio indevido, headers e sessão; testes autorizados em ambiente isolado.

CRITÉRIOS DE ACEITE

Relatório de achados com evidências reais, correções, riscos restantes e checklist por módulo.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M23.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M24 — QA, testes unitários, integração, E2E e desempenho

PROMPT M24 — QA, testes unitários, integração, E2E e desempenho

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: Cada módulo; integração final após M23.

OBJETIVO: Criar estratégia e suíte transversal com gates reais.

IMPLEMENTAÇÃO E TELAS

Unitários de domínio obrigatórios: preço/adicionais/frete/cupom, elegibilidade, horário, estados, idempotência, comissão/ledger, autorização e validação. Componentes web/mobile, integração API/Supabase local real e pgTAP RLS por operação. Contratos OpenAPI/clientes e migrações. E2E: cadastrar loja, catálogo, compra, aceite, preparo, atribuição, coleta, entrega, pagamento sandbox/reconciliação, refund e suporte; separar cenários que dependem de decisões/fornecedores. Segurança, acessibilidade, retry/offline, concorrência e regressão. Testes de carga apenas em ambiente autorizado; definir SLO/carga-alvo com proprietário antes de prometer capacidade. Não fakear teste faltante; guardar relatórios/cobertura e testes pulados com motivo. Banco de teste descartável com trava contra execução em produção.

DADOS, CONTRATOS E ACESSOS

test matrix requisito->teste->comando->resultado, pipelines, datasets somente de teste isolados.

TESTES OBRIGATÓRIOS

Executar suítes de verdade e reportar exit codes, artefatos, cobertura por módulo, flakiness e limitações de emulador/PSP.

CRITÉRIOS DE ACEITE

CI falha quando regra crítica falha; sem coverage inflado por exclusões arbitrárias; evidências reproduzíveis.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M24.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M25 — DevOps, observabilidade, deploy, backup e operação

PROMPT M25 — DevOps, observabilidade, deploy, backup e operação

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M01, M23, M24; ambiente/provedores aprovados.

OBJETIVO: Preparar produção sem publicar silenciosamente.

IMPLEMENTAÇÃO E TELAS

Configurar pipeline de lint/typecheck/test/build/scan, proteção de branches recomendada, previews sem segredos prod e ambientes separados. CI/CD para web/API/worker e builds mobile; deployment provider apenas após decisão. Staging primeiro; migrations compatíveis expand/contract, backup pré-mudança e plano de recuperação, aprovação antes de prod. Liveness/readiness reais sem detalhes secretos, logs estruturados com correlation_id e redaction, métricas de erro/latência/filas/webhooks, alertas e tracing sem PII indevida. Definir RPO/RTO e retenção com proprietário; testar restauração em ambiente separado sem sobrescrever prod. Runbooks indisponibilidade Supabase/PSP, fila parada, webhook perdido, vazamento, conta privilegiada comprometida e rollback. Banco único informado não autoriza tratar produção como sandbox.

DADOS, CONTRATOS E ACESSOS

infra versionada sem segredos, workflows, config references, dashboards por backend efetivamente contratado.

TESTES OBRIGATÓRIOS

Build clean, smoke staging, migration rehearsal, restore drill e rollback; se infraestrutura não contratada, documentar bloqueio.

CRITÉRIOS DE ACEITE

Deploy só após consentimento; checklist de acesso/dominio/TLS/secrets/backups; logs e alertas operacionais reais.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M25.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M26 — Documentação completa, integração final e GitHub

PROMPT M26 — Documentação completa, integração final e GitHub

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: Todos os módulos habilitados; M23-M25.

OBJETIVO: Entregar produto integrado com documentação e publicação aprovada.

IMPLEMENTAÇÃO E TELAS

Completar README e docs: PRD/escopo, mapa dos apps e rotas, instalação/env sem segredos, arquitetura/ADRs, ERD/dicionário/migrations, Auth/RLS/permissões, OpenAPI/exemplos sem dados reais sensíveis, manual cliente web/mobile, entregador, parceiro/gestor/rede, admin/backoffice/logística/frota, pagamentos/ledger/conciliação, marketing/suporte, privacidade/segurança/ASVS, testes/CI, deploy/rollback/backup/restore, operação/incidentes/troubleshooting, changelog/contribuição/licença pendente. Diferenciar implementado, configurado, testado, bloqueado e adiado.

Executar fluxo ponta a ponta no ambiente isolado; verificar contratos entre apps e não duplicação de regras. Secret scan no working tree/staged e histórico relevante sem imprimir segredos; achado implica parar e rotacionar, limpeza de histórico requer plano/aprovação. Verificar remote/branch e preparar resumo dos arquivos, mudanças, testes e limitações. Propor commits pequenos; executar commit/push após testes aprovados, respeitando aprovações do ambiente e publicar somente no repo alvo confirmado, sem force-push e sem

mudar visibilidade. Falta de auth GitHub: pedir conexão/credential manager seguro, nunca token no chat. Não dizer publicado sem confirmação do push e branch remoto.

DADOS, CONTRATOS E ACESSOS

docs/DELIVERY_STATUS.md e RELEASE_CHECKLIST com evidência por módulo, URLs somente quando realmente disponíveis.

TESTES OBRIGATÓRIOS

Checklist de documentação versus implementação, comandos reproduzíveis, validação de links internos, suite final e repo sem segredos.

CRITÉRIOS DE ACEITE

Release só quando gates cumpridos e pendências aceitas; componentes não configurados não podem ser chamados de sistema completo pronto para produção.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M26.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente;

dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.